

Trabajo práctico Nro 2

Filsinger Gonzalo, Perea Ignacio, Leon Parfait Manuel

*Electrónica Aplicada II 4R2, Ingeniería Electrónica, Facultad Regional Córdoba
Universidad Tecnológica Nacional*

Resumen—En este informe se abordará el desarrollo del trabajo práctico Nro 2 ej. 2, en el cual experimentaremos el conversor Analogo-Digital de la placa BluePill. Se desarrollará el segundo ejercicio del trabajo práctico.

Index Terms—BluePill, GPIO, LED, Memoria, Botones, ADC.

I. INTRODUCCIÓN

EN este informe solamente se abordará el segundo ejercicio del trabajo práctico, en que haremos uso del ADC para medir la tensión de salida de un potenciómetro (Hasta 3.3V), y temperatura de un sensor interno, según un boton que cambia entre la lectura del potenciómetro y la lectura del sensor de temperatura.

II. EJERCICIO 2: USO DE ADC

Ahora se busca controlar mediante el ADC unos leds que registran el voltaje de salida de un potenciómetro, o la opción 2, leer la temperatura del sensor interno del microcontrolador, como el ADC se configura en el PA1, usaremos el PA2 para el botón. Para esto se agregaron las siguientes definiciones en el archivo main.c:

```
// --- Registros del ADC1 ---
#define ADC1_BASE 0x40012400
#define ADC1_SR *(volatile uint32_t *) (ADC1_BASE + 0x00)
#define ADC1_CR2 *(volatile uint32_t *) (ADC1_BASE + 0x08)
#define ADC1_SMPR1 *(volatile uint32_t *) (ADC1_BASE + 0x0C)
#define ADC1_SMPR2 *(volatile uint32_t *) (ADC1_BASE + 0x10)
#define ADC1_SQR3 *(volatile uint32_t *) (ADC1_BASE + 0x34)
#define ADC1_DR *(volatile uint32_t *) (ADC1_BASE + 0x4C)

// --- Campos de Bits ---
#define RCC_AFI0EN (1 << 0)
#define RCC_IOPAEN (1 << 2)
#define RCC_IOPCEN (1 << 4)
#define RCC_ADC1EN (1 << 9)

#define GPIOC13 (1UL << 13)

#define GPIOA3 (1UL << 3) // Nuevo LED 1
#define GPIOA4 (1UL << 4) // Nuevo LED 2
#define GPIOA5 (1UL << 5) // Nuevo LED 3
#define GPIOA6 (1UL << 6) // Nuevo LED 4

#define ADC_CR2_ADON (1 << 0)
#define ADC_CR2_CAL (1 << 2)
#define ADC_CR2_EXTSEL (7 << 17)
#define ADC_CR2_EXTTRIG (1 << 20)
#define ADC_CR2_SWSTART (1 << 22)
#define ADC_CR2_TSVREFE (1 << 23)
#define ADC_SR_EOC (1 << 1)
```

Definiciones de los registros para interrupciones

- Se definió el registro **ADC1_BASE** con la dirección base del ADC, el cual se utiliza para configurar y controlar el funcionamiento del ADC.
- Se definió el registro **ADC1_SR** el cual se utiliza para verificar el estado del ADC, como por ejemplo si la conversión ha terminado o si hay algún error.
- Se definió el registro **ADC_CR2** el cual se utiliza para configurar el ADC, como por ejemplo habilitar el ADC, configurar el modo de conversión, etc.
- Se definió el registro **ADC_SMPR1** el cual se utiliza para configurar el tiempo de muestreo del ADC, lo cual es importante para obtener una conversión precisa.
- Se definió el registro **ADC_SMPR2** el cual se utiliza para configurar el tiempo de muestreo del ADC para los canales 0 a 9.
- Se definió el registro **ADC_SQR3** el cual se utiliza para configurar el canal que se va a convertir.
- Se definió el registro **ADC_DR** el cual se utiliza para leer el resultado de la conversión del ADC.

Luego se configuró que al presionar el botón se cambie el canal del ADC entre el potenciómetro y el sensor de temperatura, para esto se configuró una interrupción externa en el pin PA2, la cual se activa al detectar un flanco de subida. En la rutina de servicio de interrupción se cambia el canal del ADC entre el potenciómetro y el sensor de temperatura.

```
volatile uint8_t modo_adc = 0;

// --- ISR del Botón (Pin PA2) ---
void EXTI2_IRQHandler(void)
{
    if (EXTI_PR & (1 << 2))
    {
        EXTI_PR = (1 << 2);
        modo_adc = !modo_adc;
    }
}
```

Diagrama de conexiones para el ejercicio 2

Ahora lo nuevo que se implementa es el conversor ADC, el cual se inicializa de la siguiente forma:

```
// Inicializar ADC
ADC1_CR2 |= ADC_CR2_ADON;
delay_bucle(5000);
ADC1_CR2 |= ADC_CR2_ADON;

ADC1_CR2 |= ADC_CR2_CAL;
while(ADC1_CR2 & ADC_CR2_CAL);

ADC1_CR2 |= ADC_CR2_TSVREFE;

ADC1_CR2 |= ADC_CR2_EXTSEL | ADC_CR2_EXTTRIG;

ADC1_SMPR1 |= (0x7 << 18);
ADC1_SMPR2 |= (0x7 << 3);
```

Inicialización del ADC

Ahora se programa la función main en la cual se programa de que PIN se leerá el valor del ADC, y se muestra el resultado en los leds.

```
while (1)
{
    GPIOC_ODR ^= GPIOC13;

    if (modo_adc == 0) {
        ADC1_SQR3 = 16;
    } else {
        ADC1_SQR3 = 1;
    }

    // Iniciar conversión
    ADC1_CR2 |= ADC_CR2_SWSTART;

    // Esperar EOC (End Of Conversion)
    while ((ADC1_SR & ADC_SR_EOC) == 0);

    valor_adc = ADC1_DR;

    // Apagamos los 4 pines de la barra
    GPIOA_ODR &= ~(GPIOA6 | GPIOA5 | GPIOA4 | GPIOA3);

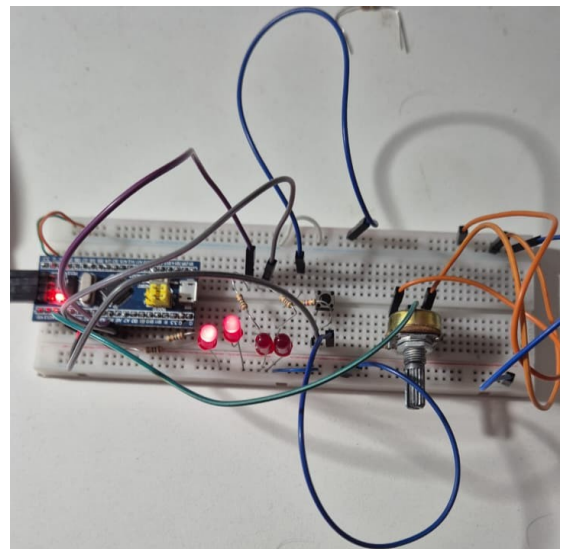
    // Encendido progresivo acumulativo
    if (valor_adc > 819) GPIOA_ODR |= GPIOA3; // Primer led
    if (valor_adc > 1638) GPIOA_ODR |= GPIOA4; // Segundo led
    if (valor_adc > 2457) GPIOA_ODR |= GPIOA5; // Tercer led
    if (valor_adc > 3276) GPIOA_ODR |= GPIOA6; // Cuarto led

    delay_bucle(30000);
}
```

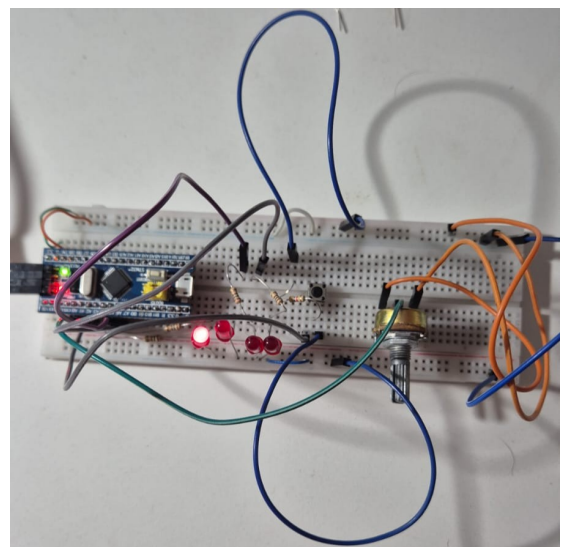
Función main para el ejercicio 2

- Se configuró el registro **ADC1_SQR3** es el registro de Secuencia Regular 3. Aquí le estamos diciendo al microcontrolador qué canal físico debe conectar al núcleo del ADC en este momento. Si es 1 lee el PA1 y si es 16 lee el sensor de temperatura interno.

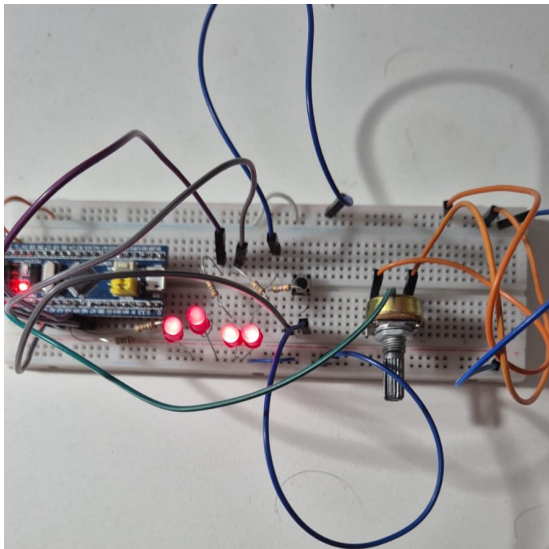
- Se configuró el registro **ADC1_CR2** el cual es Registro de Control 2 del ADC y es donde se configura su comportamiento. Al operar este registro con **ADC_CR2_SWSTART** (SoftWare Start) estamos haciendo que lea voltaje en ese momento.
- Polling es el término que se utiliza cuando se configura una espera bloqueante para que se pueda realizar la conversión. Dentro del while se encuentra **ADC1_SR & ADC_SR_EOC**, el cual opera el registro de estado del ADC con el bit de fin de conversión (EOC). Mientras se está realizando la conversión el bit EOC se mantiene en 0, y cuando la conversión termina el bit EOC se pone en 1.
- **Encendido progresivo de leds** en esta cadena de if se va encendiendo un led cada vez que el valor del ADC supera un umbral específico. Por ejemplo, si el valor del ADC es mayor a 819, se enciende el primer led, si es mayor a 1638 se enciende el segundo led, y así sucesivamente.



Funcionamiento sensor de temperatura



Funcionamiento potenciómetro con valor bajo de voltaje



Funcionamiento potenciómetro con valor alto de voltaje

La primera imagen corresponde a la lectura del sensor de temperatura, como esta es media respecto al rango de lectura del ADC, se encienden solamente dos leds. La segunda imagen corresponde a la lectura del potenciómetro con el valor bajo de voltaje, por lo que se enciende solamente un led y la tercera imagen corresponde a la lectura del potenciómetro con el valor alto de voltaje, en el cual se encuentran todos los leds encendidos.

III. CONCLUSIÓN

Se logró configurar el ADC para medir la tensión de salida de un potenciómetro y la temperatura del sensor interno del microcontrolador, mostrando los resultados en los leds.

IV. CODIGOS

IV-A. main (Ejercicio 2)

```

1  \begin{lstlisting}[language=Linker]
2  #include <stdint.h>
3
4  // --- Direcciones Base ---
5  #define RCC_BASE 0x40021000
6  #define GPIOC_BASE 0x40011000
7  #define GPIOA_BASE 0x40010800
8  #define AFIO_BASE 0x40010000
9  #define EXTI_BASE 0x40010400
10 #define NVIC_BASE 0xE000E100
11
12 // --- Registros de Periféricos ---
13 #define RCC_APB2ENR *(volatile uint32_t *) (RCC_BASE
14   + 0x18)
15 #define GPIOC_CRH *(volatile uint32_t *) (
16   GPIOC_BASE + 0x04)
17 #define GPIOA_CRL *(volatile uint32_t *) (
18   GPIOA_BASE + 0x00)
19 #define GPIOC_ODR *(volatile uint32_t *) (
20   GPIOC_BASE + 0x0C)
21 #define GPIOA_ODR *(volatile uint32_t *) (
22   GPIOA_BASE + 0x0C)
23
24 #define AFIO_EXTICR1 *(volatile uint32_t *) (
25   AFIO_BASE + 0x08)
26 #define EXTI_IMR *(volatile uint32_t *) (
27   EXTI_BASE + 0x00)
28 #define EXTI_FTSR *(volatile uint32_t *) (
29   EXTI_BASE + 0x0C)
30 #define EXTI_PR *(volatile uint32_t *) (
31   EXTI_BASE + 0x14)
32 #define NVIC_ISER0 *(volatile uint32_t *) (
33   NVIC_BASE + 0x00)
34
35 // --- Registros del ADC1 ---
36 #define ADC1_BASE 0x40012400
37 #define ADC1_SR *(volatile uint32_t *) (ADC1_BASE
38   + 0x00)
39 #define ADC1_CR2 *(volatile uint32_t *) (ADC1_BASE
40   + 0x08)
41 #define ADC1_SMPR1 *(volatile uint32_t *) (ADC1_BASE
42   + 0x0C)
43 #define ADC1_SMPR2 *(volatile uint32_t *) (ADC1_BASE
44   + 0x10)
45 #define ADC1_SQR3 *(volatile uint32_t *) (ADC1_BASE
46   + 0x34)
47 #define ADC1_DR *(volatile uint32_t *) (ADC1_BASE
48   + 0x4C)
49
50 // --- Campos de Bits ---
51 #define RCC_AFIOEN (1 << 0)
52 #define RCC_IOPAEN (1 << 2)
53 #define RCC_IOPCEN (1 << 4)
54 #define RCC_ADC1EN (1 << 9)
55
56 #define GPIOC13 (1UL << 13)
57
58 #define GPIOA3 (1UL << 3) // Nuevo LED 1
59 #define GPIOA4 (1UL << 4) // Nuevo LED 2
60 #define GPIOA5 (1UL << 5) // Nuevo LED 3
61 #define GPIOA6 (1UL << 6) // Nuevo LED 4
62
63 #define ADC_CR2_ADON (1 << 0)
64 #define ADC_CR2_CAL (1 << 2)
65 #define ADC_CR2_EXTSEL (7 << 17)
66 #define ADC_CR2_EXTTRIG (1 << 20)
67 #define ADC_CR2_SWSTART (1 << 22)
68 #define ADC_CR2_TSVREFE (1 << 23)
69 #define ADC_SR_EOC (1 << 1)

```

```

58 volatile uint8_t modo_adc = 0;
59
60 // --- ISR del Botón (Pin PA2) ---
61 void EXTI2_IRQHandler(void)
62 {
63     if (EXTI_PR & (1 << 2))
64     {
65         EXTI_PR = (1 << 2);
66         modo_adc = !modo_adc;
67     }
68 }
69
70 void delay_bucle(volatile uint32_t iteraciones) {
71     while (iteraciones > 0) iteraciones--;
72 }
73
74 void main(void)
75 {
76
77     RCC_APB2ENR |= RCC_AFIOEN | RCC_IOPAEN |
78     RCC_IOPCEN | RCC_ADC1EN;
79
80     GPIOC_CRH &= 0xFFFFFFF;
81     GPIOC_CRH |= 0x00200000;
82
83     GPIOA_CRL &= 0xF000000F;
84     GPIOA_CRL |= 0x02222800;
85     GPIOA_ODR |= (1 << 2);
86
87     // 3. AFIO y EXTI
88     AFIO_EXTICR1 &= ~(0x00000F00);
89     EXTI_IMR |= (1 << 2);
90     EXTI_FTSR |= (1 << 2);
91     NVIC_ISER0 |= (1 << 8);
92
93     // Inicializar ADC
94     ADC1_CR2 |= ADC_CR2_ADON;
95     delay_bucle(5000);
96     ADC1_CR2 |= ADC_CR2_ADON;
97
98
99     ADC1_CR2 |= ADC_CR2_CAL;
100     while(ADC1_CR2 & ADC_CR2_CAL);
101
102     ADC1_CR2 |= ADC_CR2_TSVREFE;
103
104     ADC1_CR2 |= ADC_CR2_EXTSEL | ADC_CR2_EXTTRIG;
105
106     ADC1_SMPR1 |= (0x7 << 18);
107     ADC1_SMPR2 |= (0x7 << 3);
108
109     uint16_t valor_adc = 0;
110
111     while (1)
112     {
113         GPIOC_ODR ^= GPIOC13;
114
115         if (modo_adc == 0) {
116             ADC1_SQR3 = 16;
117         } else {
118             ADC1_SQR3 = 1;
119         }
120
121         // Iniciar conversión
122         ADC1_CR2 |= ADC_CR2_SWSTART;
123
124         // Esperar EOC (End Of Conversion)
125         while ((ADC1_SR & ADC_SR_EOC) == 0);
126
127         valor_adc = ADC1_DR;
128
129         // Apagamos los 4 pines de la barra
130         GPIOA_ODR &= ~(GPIOA6 | GPIOA5 | GPIOA4 |
131         GPIOA3);

```

```

133         // Encendido progresivo acumulativo
134         if (valor_adc > 819) GPIOA_ODR |= GPIOA3;
135         // Primer nivel
136         if (valor_adc > 1638) GPIOA_ODR |= GPIOA4;
137         // Segundo nivel
138         if (valor_adc > 2457) GPIOA_ODR |= GPIOA5;
139         // Tercer nivel
140         if (valor_adc > 3276) GPIOA_ODR |= GPIOA6;
141         // Cuarto nivel (Máximo)
142
143         delay_bucle(30000);
144     }

```

Listing 1. Configuración inicial en main.c

IV-B. crt.s (Ejercicio 2)

```

1 .syntax unified
2 .cpu cortex-m3
3 .thumb
4 .extern _esstack
5
6 .section .text.manejadores // definimos la seccion
7     text.manejadores
8
9 // función básica para manejar los punteros
10 .thumb_func
11 .weak Default_Handler
12 Default_Handler:
13     b . /* Bucle infinito */
14
15 // definir los vectores ejemplo
16 .weak NMI_Handler
17 .thumb_set NMI_Handler, Default_Handler
18 .weak SysTick_Handler
19 .thumb_set SysTick_Handler, Default_Handler
20 .weak HardFault_Handler
21 .thumb_set HardFault_Handler, Default_Handler
22 .weak RCC_IRQHandler
23 .thumb_set RCC_IRQHandler, Default_Handler
24 .weak EXTI0_IRQHandler
25 .thumb_set EXTI0_IRQHandler, Default_Handler
26 .weak EXTI1_IRQHandler
27 .thumb_set EXTI1_IRQHandler, Default_Handler
28 .weak EXTI2_IRQHandler
29 .thumb_set EXTI2_IRQHandler, Default_Handler
30 .weak EXTI3_IRQHandler
31 .thumb_set EXTI3_IRQHandler, Default_Handler
32
33 // vectores
34 .section .isr_vector,"a",%progbits
35 .word _esstack
36 .word _reset
37 .word NMI_Handler /* 2: NMI (Non-Maskable
38     Interrupt) */
39 .word HardFault_Handler /* 3: HardFault (falla
40     grave) */
41 .word 0 /* 4: MemManage (falla de
42     memoria) */
43 .word 0 /* 5: BusFault (falla de
44     bus) */
45 .word 0 /* 6: UsageFault (falla
46     de uso) */
47 .word 0 /* 7: Reservado */
48 .word 0 /* 8: Reservado */
49 .word 0 /* 9: Reservado */
50 .word 0 /* 10: Reservado */
51 .word 0 /* 11: SVCALL (Llamada al
52     Supervisor) */
53 .word 0 /* 12: Debug Monitor */
54 .word 0 /* 13: Reservado */
55 .word 0 /* 14: PendSV (Interrupció
56     n pendiente del sistema) */

```

```

49 .word SysTick_Handler      /* 15: El handler del
    SysTick */
50 .word 0                    /* IRQ0: WWDG */
51 .word 0                    /* IRQ1: PVD */
52 .word 0                    /* IRQ2: TAMPER */
53 .word 0                    /* IRQ3: RTC */
54 .word 0                    /* IRQ4: FLASH */
55 .word RCC_IRQHandler       /* IRQ5: RCC */
56 .word EXTI0_IRQHandler     /* IRQ6: EXTI0 */
57 .word EXTI1_IRQHandler     /* IRQ7: EXTI1 (PA1) */
58 .word EXTI2_IRQHandler     /* IRQ8: EXTI2 (PA2) */
59 .word EXTI3_IRQHandler     /* IRQ9: EXTI3 */
60
61 // declaramos la función _reset dentro de la seccion
    text.reset
62 .section .text.reset
63 .thumb_func
64 .global _reset
65 _reset:
66     bl main
67     b .

```